

SUMMER INTERNSHIP REPORT ON
PYTHON FULL STACK DEVELOPER VIRTUAL
INTERNSHIP

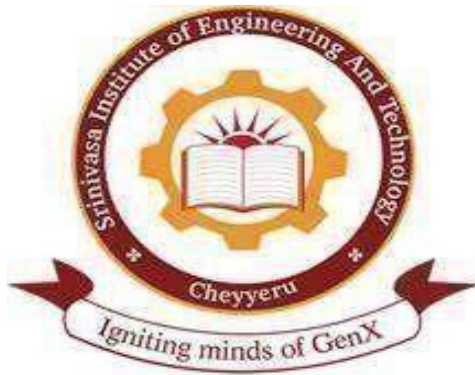
in partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING



Submitted by

HEMA LAKSHMI BOKKA

226N1A0512

SRINIVASA INSTITUTE OF ENGINEERING & TECHNOLOGY

(UGC– Autonomous Institution)

(Approved by AICTE, permanently affiliated to JNTUK, Kakinada) (ISO 9001:2015 Certified Institute)

(Accredited by NAAC with 'A' Grade) (Recognized by UGC under sections 2(f) & 12(B))

NH-216, Cheyyeru (v), Amalapuram-533222

2025-2026

SRINIVASA INSTITUTE OF ENGINEERING AND TECHNOLOGY

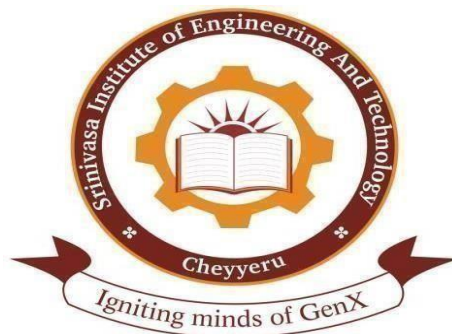
(UGC-Autonomous)

(Approved by AICTE, New Delhi & Permanently Affiliated to JNTUK, Kakinada)

(An ISO 9001:2015 Certified Institute, Accredited by NAAC with 'A' Grade) NH-216,

Amalapuram-Kakinada Highway, Cheyyeru (V), Amalapuram.

COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that **HEMA LAKSHMI BOKKA** Reg. No. **226N1A0512** has completed his/her Internship in **EduSkills** on **Python Fullstack Developer** as a part of partial fulfillment of the requirement for the Degree of **Bachelor of Technology** in the **Department of Computer Science and Engineering** for the academic year 2025-2026.

Signature of the Mentor

Head of the Department

Principal



EduSkills
ACADEMY



CERTIFICATE

— OF COMPLETION —

Awarded to

Hema Lakshmi Bokka

has successfully completed

Python Full Stack

Date : 28 May 2025

C. No.:ec6309b8044a695ade2e793954b61b3b

Chief Technology Officer (CTO)
EduSkills





अखिल भारतीय तकनीकी शिक्षा परिषद्
All India Council for Technical Education



Certificate of Virtual Internship

This is to certify that

Hema Lakshmi Bokka

Srinivasa Institute of Engineering and Technology (Autonomous)

has successfully completed 10 weeks

Python Full Stack Developer Virtual Internship

During April - June 2025

We wish him / her all the best for the future endeavours

Supported by  **EduSkills**
ACADEMY

Shri Buddha Chandrasekhar
Chief Coordinating Officer (CCO)
NEAT Cell, AICTE

Dr. Satya Ranjan Biswal
Chief Technology Officer (CTO)
EduSkills



Certificate ID : dde66bae1682750c67c3afa4bbb66e46

Student ID : STU66a118e792eea1721833703



GRADE- O (Outstanding): 90-100 | E (Excellent): 80-89 | A (Very Good): 70-79 | B (Good): 60-69 | C (Fair): 50-59 | D (Average): 40-49 | P (Pass): 30-39 | F (Fail): Below 30

ACKNOWLEDGEMENT

I would like to extend my sincere gratitude to **EduSkills** team and my mentors for their valuable support throughout my PYTHON FULLSTACK DEVELOPER Virtual Internship. This opportunity provided me with practical insights into encryptions and network engineering, significantly enhancing my technical skills and professional growth. Your leadership was key to making this learning experience truly impactful.

I sincerely appreciate **EduSkills** for the comprehensive curriculum in my PYTHON FULLSTACK Virtual Internship. It provided valuable insights and significantly enhanced my technical capabilities for future challenges.

Our sincere gratitude goes to **Mrs. Y.Sowjanya**, our Mentor, whose constant support, valuable feedback, and motivating presence steered us through the challenges we encountered during the project. Her leadership played a critical role in the successful completion of our internship.

I am deeply indebted to **Dr. Saipriya V**, M Tech, Ph.D., Head of Department and Professor of CSE, for her guidance and for ensuring we had access to the necessary resources and support throughout the internship. Her encouragement has been a driving force in our progress.

My sincere thanks also go to **DR. K. Suresh Babu**, Principal, for providing us with the opportunity to embark on this journey, as well as for the continuous support extended during this period.

Finally, I would like to express my appreciation to our College Management, faculty, lab technicians, non-teaching staff, and friends, who have played an essential role in helping us complete the internship. Their timely support, both direct and indirect, contributed greatly to our success.

Hema Lakshmi Bokka

226N1A0512

ABSTRACT

In today's fast-paced digital landscape, full stack development has become a cornerstone for building dynamic, scalable, and responsive web applications. Our Python Full Stack Internship focused on mastering both frontend and backend technologies using Python as the core programming language.

This program emphasized hands-on experience with frameworks like Django and Flask for backend development, while integrating HTML, CSS, JavaScript, and React for frontend design. Participants gained proficiency in RESTful API creation, database management with MySQL and MongoDB, and deployment strategies using cloud platforms and version control systems like Git.

The internship addressed real-world challenges such as user authentication, data validation, and responsive UI design, ensuring that learners could secure and efficient web applications. Emphasis was placed on modular coding, debugging techniques, and performance optimization to meet industry standards.

By the end of the program, participants were equipped to develop full-fledged web solutions- from designing intuitive user interfaces to managing complex server-side logic- bridging the gap between client needs and technical implementation. This comprehensive training empowers developers to thrive in full stack roles and contribute meaningfully to modern software teams.

Organization Information

EduSkills has launched a comprehensive Python Full Stack Development training initiative aimed at equipping students and professionals with industry-relevant skills. This program focuses on core full stack concepts, including frontend and backend development, database integration, and deployment strategies. It is designed to prepare participants to tackle real- world web development projects.

DESCRIPTION OF INTERNSHIP

The Python Full Stack Internship offered by EduSkills provides immersive, hands-on experience in building complete web applications using both frontend and backend technologies. This program is structured to cover essential concepts in full stack development, including user interface design, server-side logic, database integration, and deployment strategies.

Participants gain practical exposure to:

- Frontend Technologies: HTML, CSS, JavaScript, and React for creating responsive and interactive user interfaces.
- Backend Frameworks: Python-based frameworks like Django and Flask for managing server-side operations and business logic.
- Development Tools: Git, GitHub, RESTful APIs, and cloud platforms for version control, collaboration, and deployment.

As the demand for full stack developers continues to grow, EduSkills' internship aligns with industry best practices, ensuring participants develop proficiency in modular coding, debugging, and scalable architecture. The program culminates in a capstone project where learners apply their skills to build a real-world web application, reinforcing their understanding and boosting their employability.

INDEX

Week	Modules	Pg.no
1	Certificates	I-IV
2	Acknowledgement	V
3	Abstract	VI
4	Description Of Internship	VII
5	Introduction to Full Stack Development and Web Architecture	1-3
6	HTML, CSS, and Responsive Design Principles	4-6
7	JavaScript Fundamentals and DOM Manipulation	7-10
8	React Basics and Component-Based UI Development	11-14
9	Python Programming Essentials and Environment Setup	15-17
10	Backend Development with Django and Flask	18-21
11	Database Integration with MySQL and MongoDB	22-24
12	RESTful APIs and Connecting Frontend to Backend	25-27
13	Version control with Git and Development Strategies	28
14	Capstone Project Implementation and Final Recap	31
15	Flow of Python Full Stack Developer	34
16	Result & discussions (Technology stack)	35
16	conclusion	36

Week 1

Introduction to Full Stack Development and Web Architecture

Topic Covered: Understanding Full Stack Roles, Web Architecture, and Development Lifecycle

Description:

Full Stack Development refers to the ability to work on both the frontend (client-side) and backend (server-side) of web applications. A Full Stack Developer is proficient in designing user interfaces, managing databases, writing server logic, and deploying applications.

This week introduces the foundational concepts of full stack development, the structure of modern web applications, and the technologies involved in building scalable, responsive, and secure web solutions.

Key Areas Covered:

- Definition and Scope of Full Stack Development: Understanding what full stack means and the responsibilities of a full stack developer.
- Frontend vs Backend: Differentiating between client-side and server-side technologies and their roles in web applications.
- Web Architecture Overview: Exploring the 3-tier architecture: Presentation Layer (HTML, CSS, JS), Logic Layer (Python, Django, Flask), Data Layer (MySQL, MongoDB)
- Development Lifecycle: Introduction to planning, designing, coding, testing, and deploying web applications.
- Popular Full Stack Technologies: Overview of tools and frameworks such as: (a)Frontend- HTML, CSS, JavaScript, React (b)Backend- Python, Django, Flask (c)Database- MySQL, MongoDB (d)Version control- Git, GitHub (e)Deployment- Heroku, Netlify, AWS

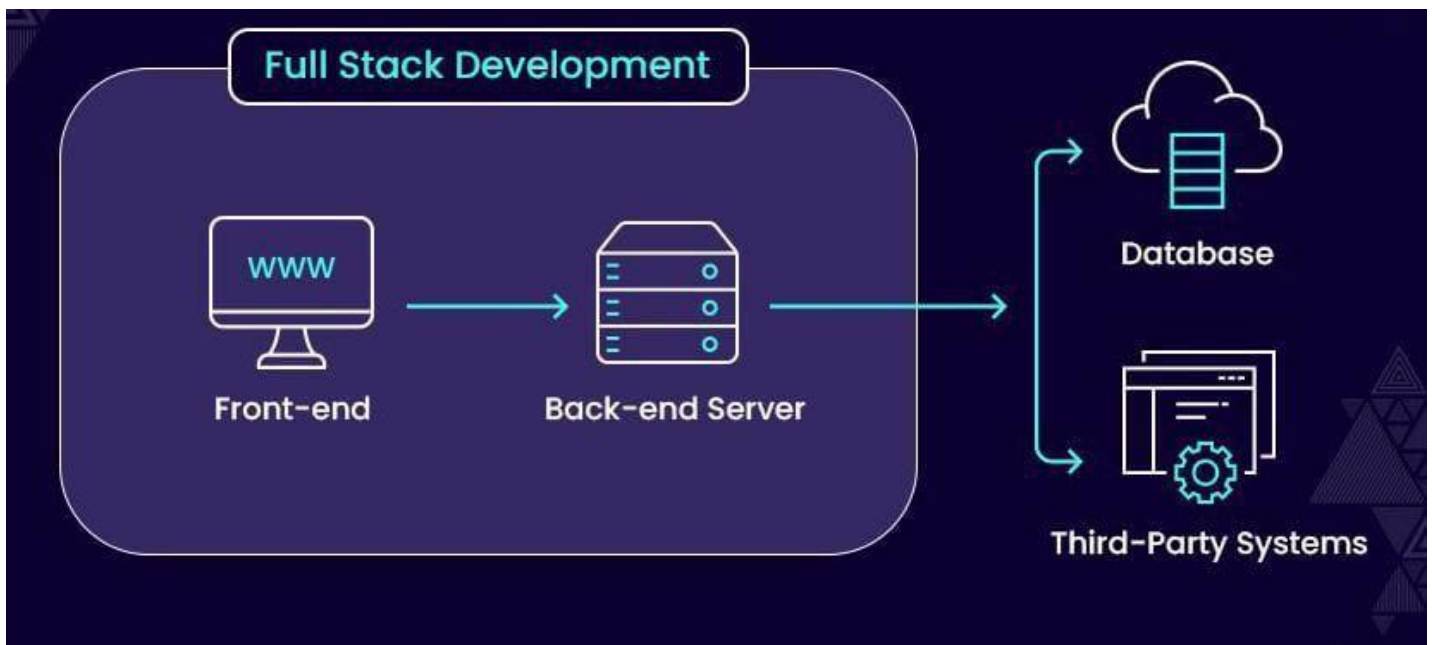
Why Full Stack Development?

- Enables end-to-end control over web application development.
- Enhances problem-solving by understanding both client and server perspectives.

- Increases employability by mastering multiple technologies.
- Facilitates faster development cycles and better collaboration across teams.

Disciplines Involved:

- Software Engineering
- Computer Science
- Cloud computing



• Fig 1.1 – Architecture of Full Stack Development

Advantages of Python Full Stack Development:

- **End-to-end Development Capability:** Full stack developers can handle both frontend and backend tasks, enabling complete control over application architecture and faster development cycles.
- **High Demand in the Industry:** Python full stack developers are highly sought after due to their versatility and ability to work across multiple layers of technology.
- **Rapid Prototyping and Deployment:** Python's simplicity and rich ecosystem allow developers to build and deploy applications quickly, making it ideal for startups and agile teams. (Flask, NumPy, Pandas) and a vibrant developer community, which accelerates learning and problem-solving.

- **Scalability and Maintainability:** Modular coding practices and MVC architecture in frameworks like Django ensure that applications are scalable and easy to maintain.
- **Cross-Platform Compatibility:** Python applications can run on various operating systems, making them flexible and adaptable to different environments.
- **Integration with modern tools:** Full stack development with python supports integration with REST APIs, cloud platforms, and DevOps tools, enhancing deployment and collaboration.
- **Enhanced Career Opportunities:** Mastery of full stack developments opens doors to roles such as Full Stack Developer, Backend Engineer, Web Developer, and Software Engineer.

Real World Applications:

- Developing social media applications
- Designing content management systems (CMS)
- Implementing secure login and user authentication
- UI/UX Design
- Database Management
- Creating e-commerce platforms
- Building dynamic websites and dashboards

WEEK 2

HTML, CSS, And Responsive Design Principles

Topic Covered: Structuring Web Pages and Styling User Interfaces

Description:

This week focuses on the *frontend foundation* of web development-HTML and CSS. These technologies are essential for creating the structure and visual presentation of web pages.

HTML(Hyper Text Markup Language) defines the content and layout, while CSS(Cascading Style Sheets) controls the design, colors, fonts, and responsiveness.

Participants learn how to build clean, semantic web pages and apply styling rules to enhance user experience across devices.

Why HTML & CSS?

- They form the *visual and structural backbone* of every website.
- Enable developers to create *user-friendly, accessible, and visually appealing* interfaces.
- Responsive design ensures websites adapt to *different screen sizes and devices*, improving usability.
-

Key Areas Covered:

- **HTML Basics:** -Tags, elements, attributes
 - Page structure: <head>, <body>, <header>, <footer>, <section>
 - Lists, tables, forms, and multimedia embedding
- **CSS Fundamentals:** -Selectors, properties, and values
 - Inline, internal, and external stylesheets
 - Box model, positioning, and layout techniques

- **Responsive Design:** -Media queries and breakpoints
-Flexbox and Grid layout systems
-Mobile-first design principles
- **Semantic HTML:** -Using meaningful tags to improve accessibility and SEO

Tools Introduced:

1. Code Editors: VS Code, sublime text
2. Live Preview Extensions
3. Browser Developer Tools for inspecting and debugging styles

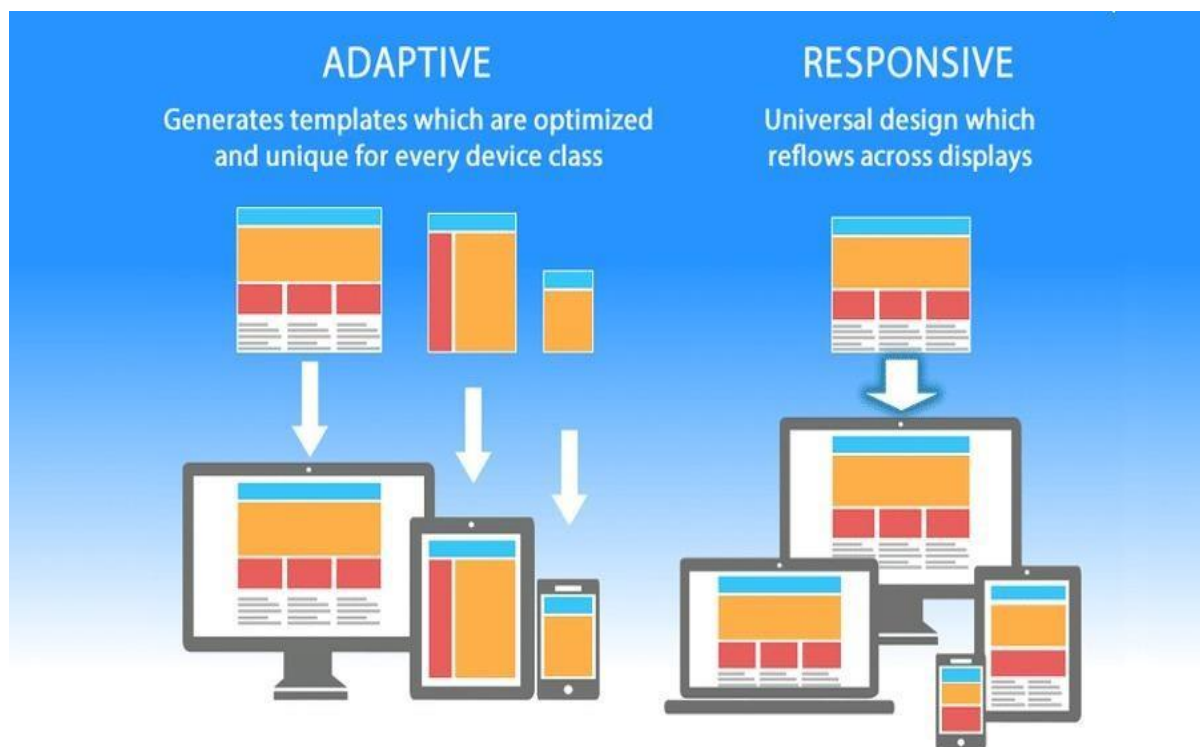


Fig. 2.1. Responsive Web Design Architecture

Real-World Applications:

- Designing landing pages and dashboards
- Creating forms for user input
- Building responsive layouts for mobile and desktop
- Styling components in react and other frontend frameworks

Week 3

JavaScript Fundamentals and DOM Manipulation Topic

Covered: Programming Logic and Dynamic web interaction

Description:

JavaScript is the backbone of dynamic web behavior. It allows developers to create interactive elements,

respond to user actions, and manipulate the structure and content of web pages in real time. This week introduces the core syntax and logic of JavaScript, along with the powerful concept of DOM (Document Object Model) manipulation, which enables dynamic updates to HTML and CSS.

Participants learn how to write scripts that respond to events, validate input, and enhance user experience through interactive design.

Key Areas Covered:

JavaScript Basics:

- Variables (var, let, const)
- Data types and operators
- Control flow: if, else, switch, loops

Function and Scope:

- Declaring and invoking functions
- Parameters, return values
- Global vs local scope

Events and Event Handling:

- Onclick, onchange, onmouseover, etc.
- Event listeners and callbacks

DOM Manipulation:

- Accessing elements with `getElementById`, `querySelector`.
- Changing content and styles dynamically.
- Creating, appending, and removing elements.

Form Validation:

- Checking user input before submission.
- Displaying error messages and feedback.

Why JavaScript?

- Enables real-time interactivity on websites.
- Powers features like dropdowns, modals, sliders, and form validation.
- Essential for frontend frameworks like React, Angular, and Vue.
- Forms the core of client-side scripting, making web pages responsive and engaging.

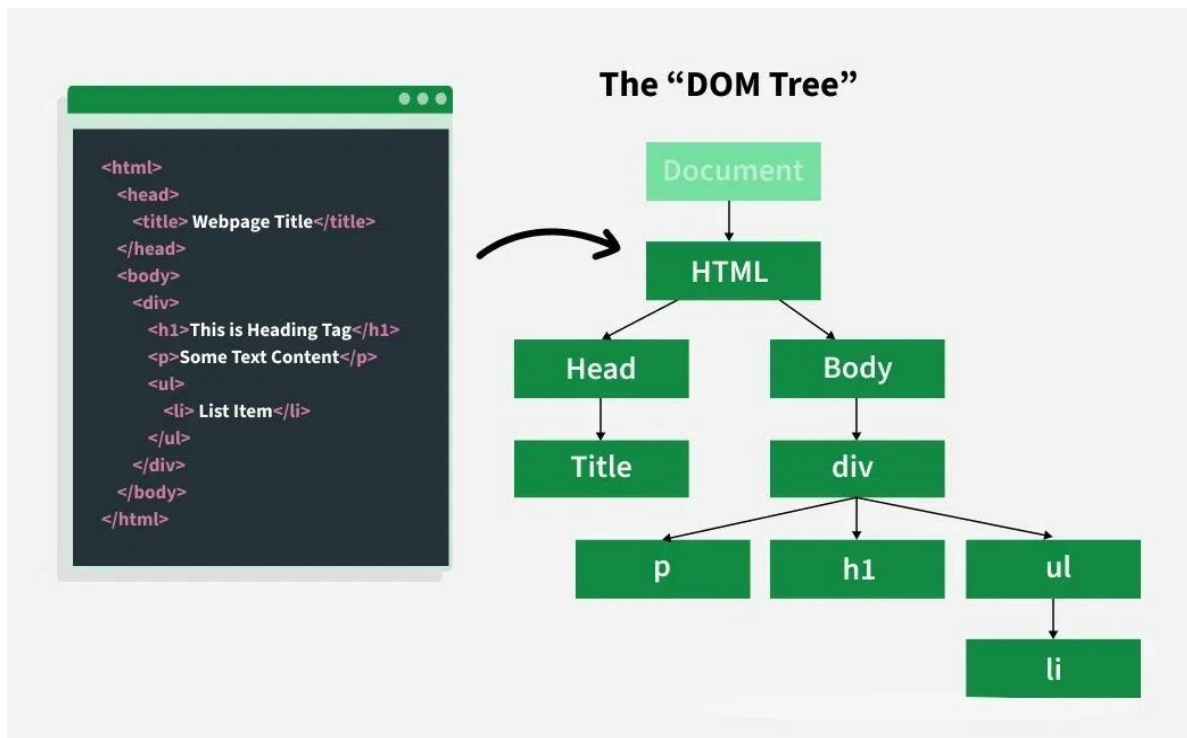


Fig. 3.1. JavaScript DOM Manipulation overview

Tools Introduced:

- **Browser Console for debugging**
- Live Server extension for real-time preview
- JSFiddle / CodePen for quick prototyping

Advantages of JavaScript and DOM Manipulation:

- **Dynamic Interactivity**

JavaScript enables real-time updates to web pages without reloading, enhancing user experience through animations, modals, and form validation.

- **Client-Side Execution**

JavaScript runs directly in the browser, reducing server load and improving performance.

- **DOM Control**

Developers can access and modify HTML elements, attributes, and styles dynamically using DOM methods like `getElementById`, `querySelector`, and `addEventListener`.

- **Cross-Browser Compatibility**

JavaScript is supported by all major browsers, making it a reliable choice for frontend development

- **Enhanced User Experience**

Real-time feedback, dynamic content updates, and smooth transitions contribute to a more engaging and responsive UI.

- **Foundation for Frameworks**

JavaScript powers modern libraries and frameworks like React, Angular, and Vue, which rely heavily on DOM manipulation

- **Event-Driven Programming**

DOM manipulation allows developers to respond to user actions (clicks, inputs, hovers) and create interactive interfaces

Real-World Applications:

- Interactive forms and validation
- Dynamic content updates without page reload
- Responsive UI elements like tabs, accordions, and carousels
- Building the logic behind single-page applications (SPAs)

Week 4

React Basics and Component-Based UI Development

Topic Covered: Building Modular and Interactive Frontend Interfaces

Description:

React is a powerful JavaScript library developed by Facebook for building dynamic and responsive user interfaces. It follows a component-based architecture, allowing developers to break down complex UIs into reusable, manageable pieces. This week introduces the fundamentals of React, including JSX syntax, props, state management, and component lifecycle.

Participants learn how to create interactive web applications using React's declarative approach and virtual DOM efficiency.

Key Areas Covered:

Introduction to React

- What is React and why use it
- SPA (Single Page Application) concept

JSX Syntax

- Writing HTML-like code in JavaScript
- Embedding expressions and styling

Components

- Functional vs Class components
- Props (data passed to components)
- State (internal data management)

Event Handling in React

- Handling user interactions with onClick, onChange, etc

Conditional Rendering

- Displaying content based on logic and state

React Developer Tools

- Debugging and inspecting component hierarchy

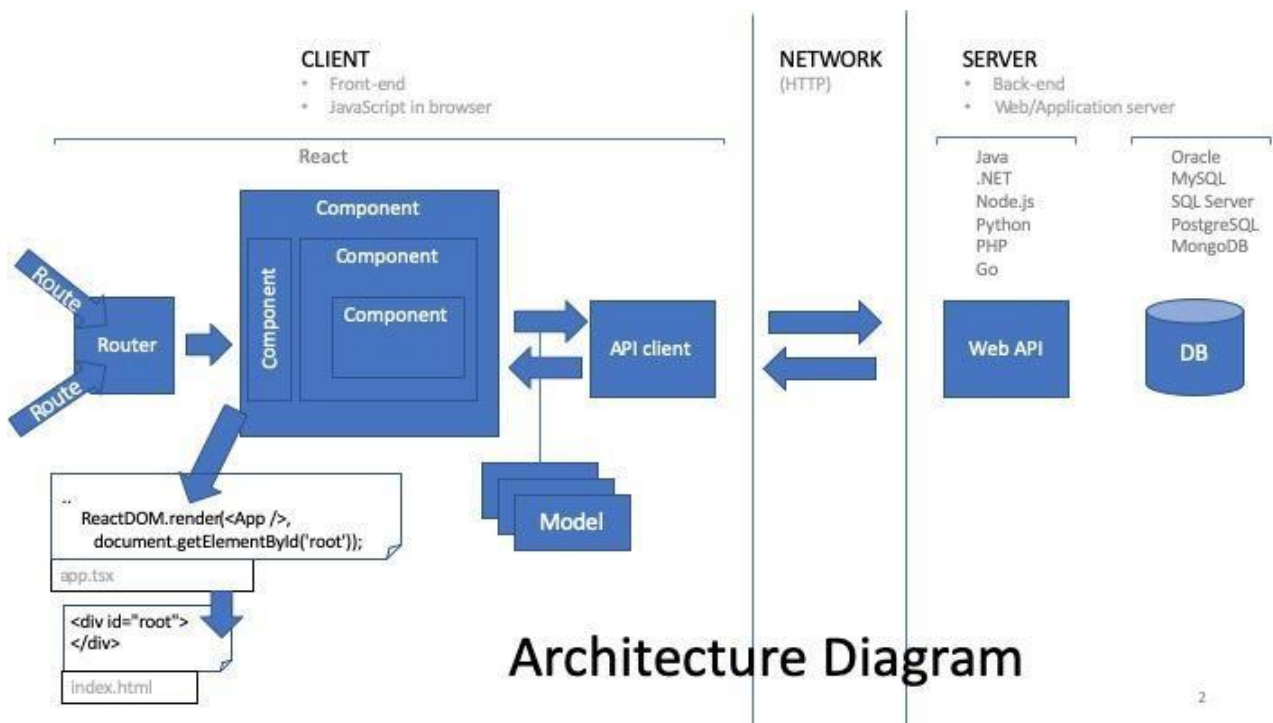
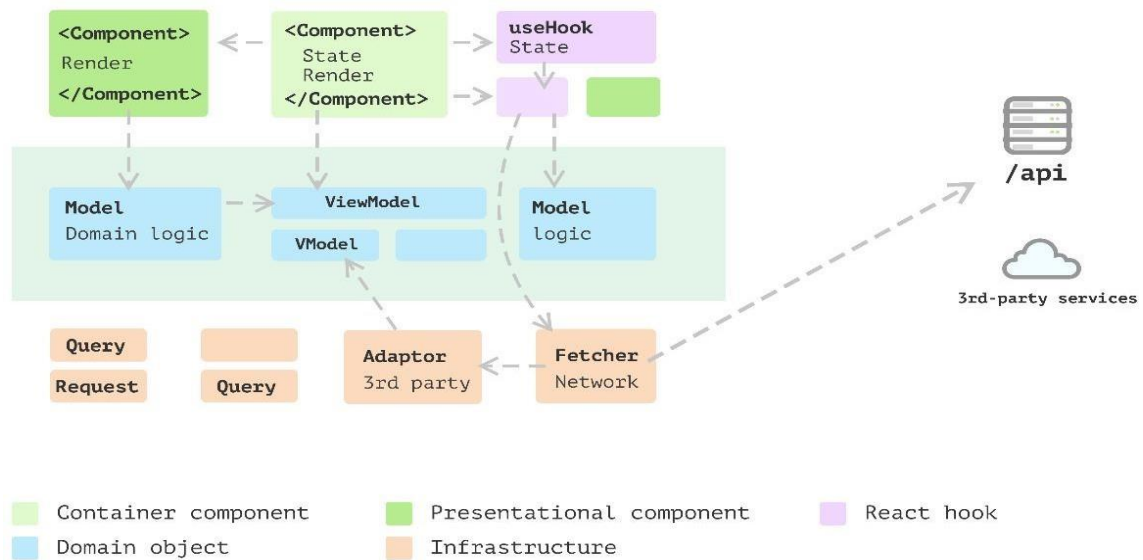


Fig.4.1 React Component-Based Architecture

Advantages of React:

- **Component Reusability**

Speeds up development and ensures consistency across the UI

- **Virtual DOM**

Improves performance by minimizing direct DOM manipulation.

- **Declarative Syntax**

Makes code more readable and easier to debug

- **Strong Community and Ecosystem**

Rich library support and frequent updates

- **Integration Friendly**

Can be used with other libraries and frameworks

- **Scalable Architecture**

Ideal for building large-scale applications with modular design

Tool Introduced:

- Node.js & npm for package management
- Create React App for project scaffolding
- React DevTools for browser inspection

Real-World Applications:

- Building dashboards and admin panels
- Designing interactive forms and modals
- Creating dynamic content feeds (e.g., social media, blogs)
- Developing scalable frontend architecture for enterprise apps

Week 5

Python Programming Essentials and Environment

Topic Covered: Core Python Concepts and Development Tools **Description:**

Python is a versatile, beginner-friendly programming language widely used in web development,

data science, automation, and AI. This week introduces the foundational syntax and logic of python, along with the tools required to set up a full stack development environment.

Participants learn how to write clean, efficient Python code and configure their systems for backend development using frameworks like Django and Flask.

Key Areas Covered:

- **Python Basics**

Variables, data types, and operators Control structures: if, else, while, for loops Functions, parameters, and return values

- **Data Structure**

Lists, tuples, dictionaries, and sets Iteration and manipulation techniques

- **Error Handling**

Try-except blocks

Debugging and exception management

- **Environment Setup**

Installing Python and pip and requirements.txt

- **IDE and Tools**

VS Code, PyCharm Terminal/Command Line basics Git and GitHub initialization

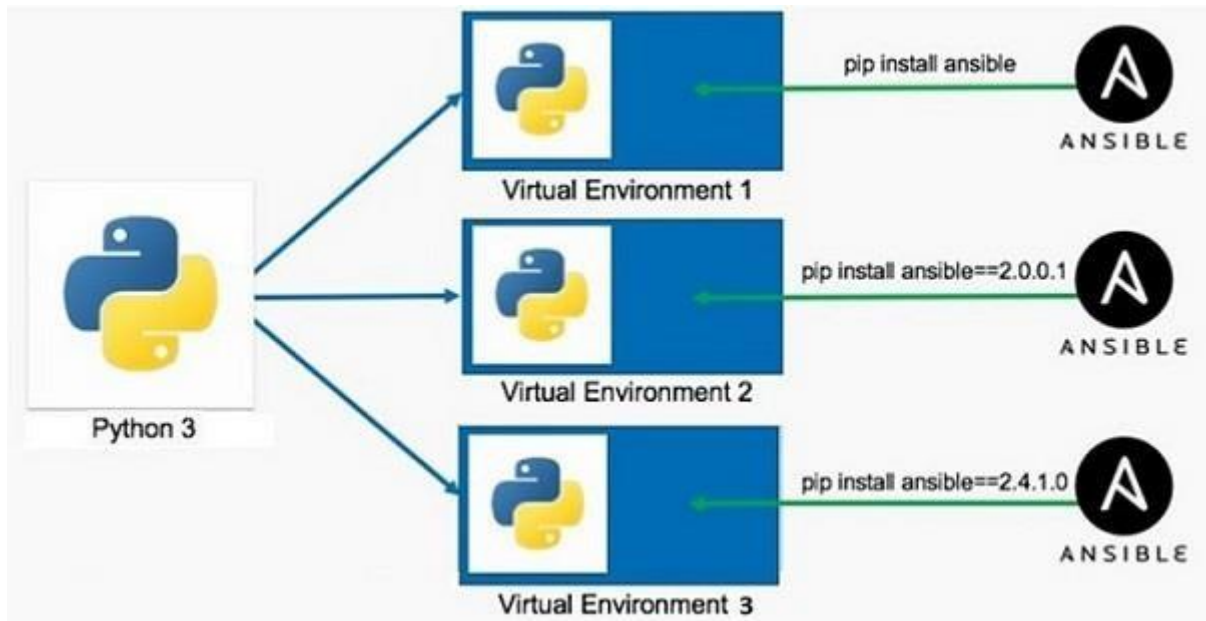


Fig. 5.1 Python Development Environment Setup

Advantages of Python in Full Stack Development:

- **Readable and Concise Syntax:** Makes it easy to learn and maintain code
- **Extensive Libraries and Framework:** Django and Flask simplify backend development
- **Cross-Platform Compatibility:** Python runs on windows, macOS, and Linux
- **Strong Community Support:** Abundant tutorials, documentation, and forums
- **Rapid Development:** Ideal for prototyping and building scalable applications
- **Integration Friendly:** Easily connects with databases, APIs, and frontend technologies.

Tools Introduced:

- Python Interpreter
- VS Code / PyCharm IDE

- Pip for package management
- Git & GitHub for version control
- Virtual Environment (venv)

Real-World Applications:

- Backend logic for web applications
- Data processing and automation
- API development and integration
- Server-side validation and business rules

WEEK 6

Backend Development with Django and Flask

Topic Covered: Server-Side Logic, Routing, and Data Handling

Description:

Backend development focuses on the server-side logic that powers web applications. This week

introduces two popular Python frameworks-Django and Flask-used to build robust, scalable, and secure backends. Participants learn how to handle HTTP requests, define routes, manage databases, and connect frontend interfaces to backend logic.

Django offers a high-level, batteries-included approach, while Flask provides lightweight flexibility for custom applications.

Key Areas Covered:

Introduction to Backend Development

- Role of backend in full stack architecture
- Client-server communication using HTTP

Flask Basics

- Setting up a Flask app
- Routing with `@app.route()`
- Handling GET and POST requests
- Rendering templates with Jinja2

Django Fundamentals

- Project and app structure
- URL routing and views
- Models and ORM (Object-Relational Mapping)
- Admin interface and migrations

Template Rendering

- Connecting backend logic to HTML templates
- Passing data from views to templates

Form Handling and Validation

- Creating forms
- Validating user input
- Displaying feedback

Advantages of Django and Flask:

- **Rapid Development**

Django's built-in features (admin panel, ORM, authentication) speed up development

- **Flexibility and Control**

Flask allows custom setups and minimal overhead for small projects

- **Secure and Scalable**

Django include security features like CSRF protection, user authentication, and input validation

- **Modular Architecture**

Both frameworks support reusable components and clean code organization

- **Strong Community Support**

Extensive documentation and active developer communities

- **Database Integration**

Built-in ORM simplifies database operations and migrations.

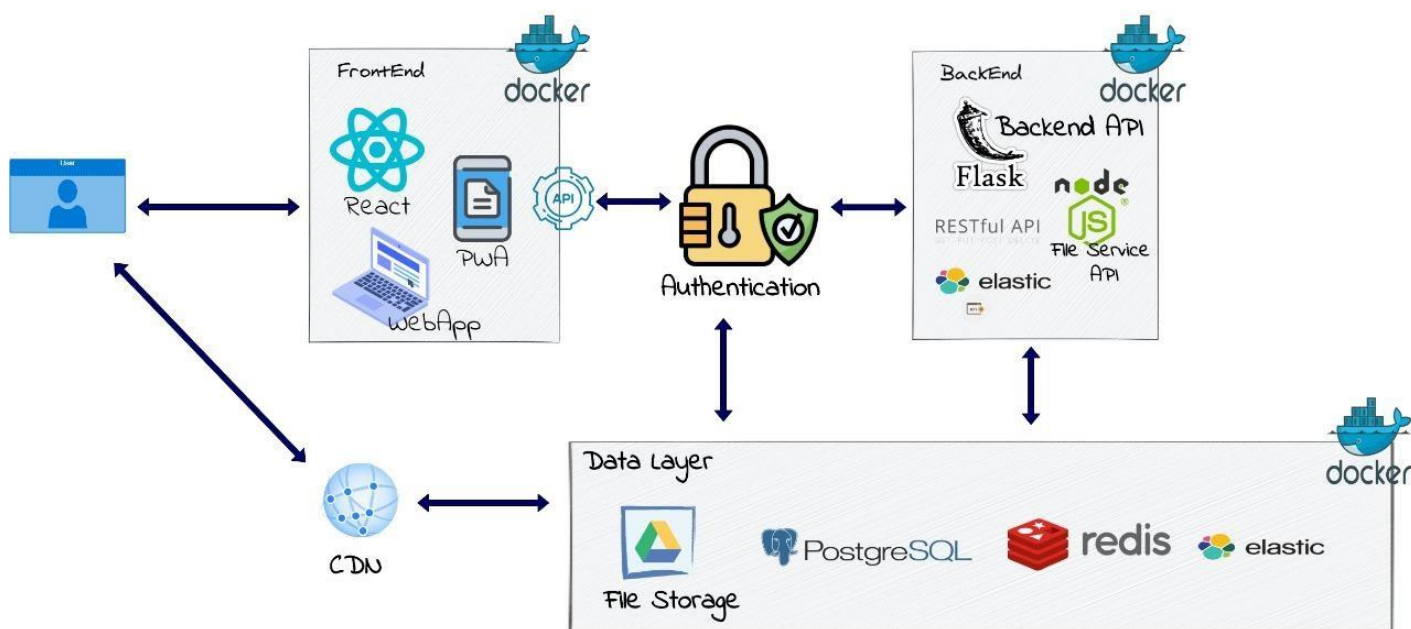
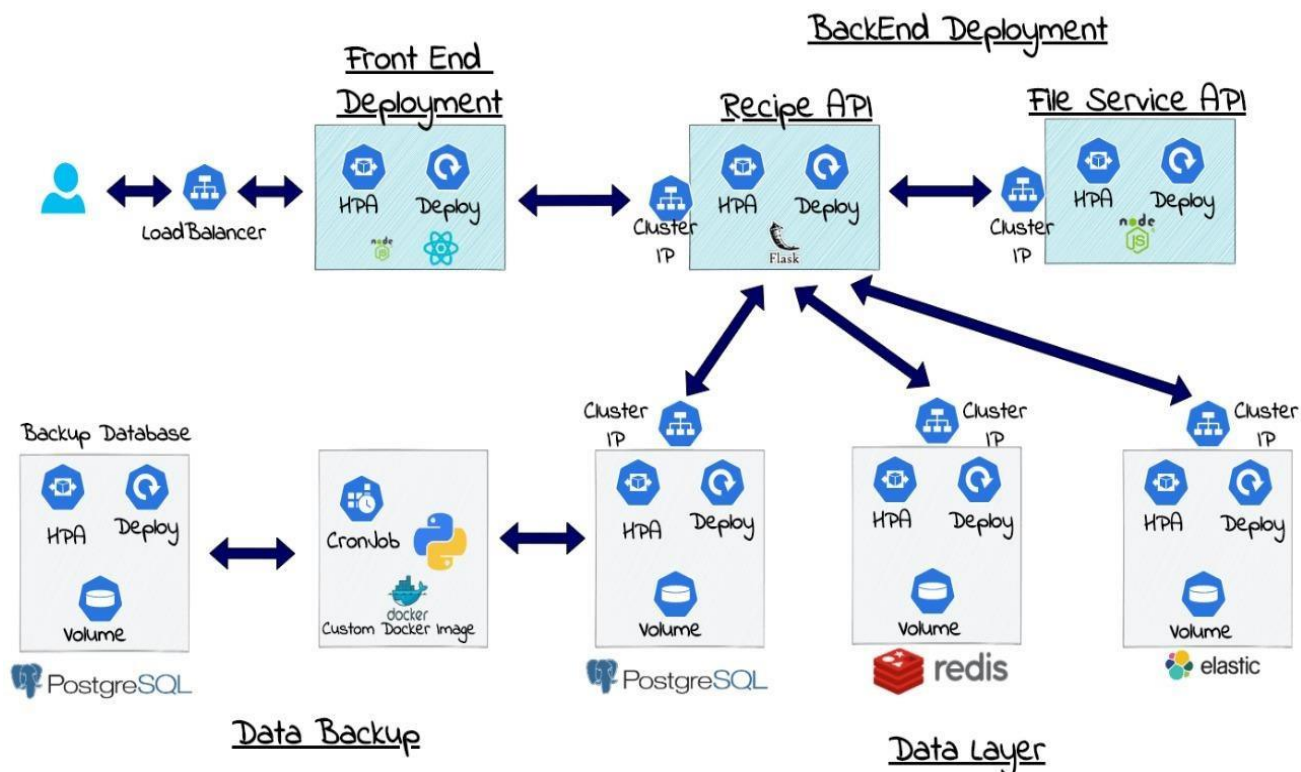


Fig 6.1 & Fig 6.2 Django vs Flask Backend Architecture

Tools Introduced:

- Flask CLI and Jinja2
- Django Admin and ORM
- SQLite / MySQL for backend storage
- Postman for API testing

Real-World Applications:

- Building login and registration systems
- Creating dashboards and admin panels
- Developing RESTful APIs
- Managing user data and form submissions
- Connecting frontend forms to backend logic

Week 7

Database Integration with MySQL and MongoDB

Topic Covered: Structured and Unstructured Data Management in Full Stack Applications

Description:

Databases are the backbone of dynamic web applications, storing and retrieving data efficiently. This week introduces two major types of databases:

- **MySQL** (Relational Database Management System)
- **MongoDB** (NoSQL Document Database)

Participants learn how to connect backend applications to databases, perform CRUD operations, and choose the right database based on project needs.

Key Areas Covered:

Relational vs NoSQL Databases

- Structured vs unstructured data
- Tables vs collections
- SQL vs JSON-like queries

MySQL Integration

- Creating databases and tables
- Writing SQL queries (SELECT, INSERT, UPDATE, DELETE)
- Connecting with Django ORM or Flask-SQLAlchemy

MongoDB Integration

- Collections and documents
- CRUD operations using PyMongo
- Schema flexibility and indexing

Connecting to Backend

- Using Environment variables for credentials
- Performing queries from Python backend
- Displaying data on frontend via templates or APIs

Advantages of MySQL and MongoDB:

MySQL

- Strong data integrity and ACID compliance
- Ideal for structured, transactional applications
- Mature ecosystem and widespread support

MongoDB

- Flexible schema for evolving data models
- High performance for large-scale, unstructured data
- Easy to scale horizontally

General Benefits

- Enable persistent data storage
- Support real-time data access and updates
- Power features like login systems, dashboards, and analytics

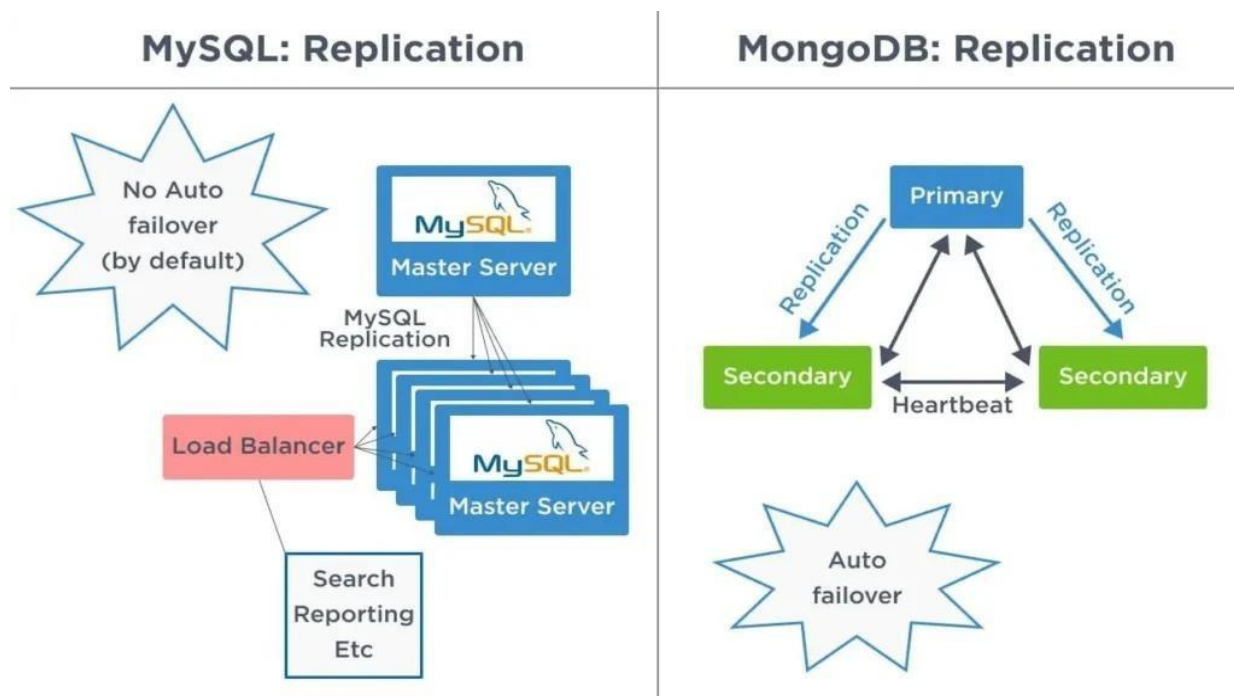


Fig 7.1 MySQL vs MongoDB Architecture

Tools Introduced:

- MySQL Workbench / phpMyAdmin
- MongoDB Compass / Atlas
- Django ORM / Flask-SQLAlchemy / PyMongo
- Postman for testing database-connected APIs

Real-World Applications:

- Storing user profiles and login credentials
- Managing product catalogs and inventory
- Logging user activity and analytics
- Powering dashboards and admin panels

Week 8

RESTful APIs and Connecting Frontend to Backend

Topic Covered: Data Exchanged Between Client and Server Using REST Architecture

Description:

This week focuses on integrating the frontend and backend of a full stack application using RESTful APIs. REST (Representational State Transfer) is a widely used architectural style that enables communication between client and server through standardized HTTP methods.

Participants learn how to create, consume, and test APIs to enable dynamic data exchange and modular application design.

Key Areas Covered:

Understanding REST Architecture

- What is an API?
- REST vs SOAP vs GraphQL (brief comparison)
- Client-server model and stateless communication

HTTP Methods and Status Codes

- GET, POST, PUT, DELETE
- Common status codes: 200, 404, 500

Consuming APIs (Frontend)

- Using fetch() and axios in JavaScript/React
- Displaying dynamic data from backend
- Handling errors and loading states

Creating APIs (Backend)

- Building endpoints in Flask and Django
- Serializing data with Django REST Framework (DRF)
- Returning JSON responses

Testing and Debugging

- Using Postman to test endpoints
- Inspecting network requests in browser dev tools

Advantages of REST API Integration:

- **Modular Architecture**-Separates frontend and backend logic for better scalability
- **Standardized Communication**-HTTP methods and status codes ensure predictable behavior
- **Reusability and Maintainability**-APIs can be reused across multiple projects and teams
- **Cross-Platform Compatibility**-APIs can be consumed by web, mobile, or desktop clients

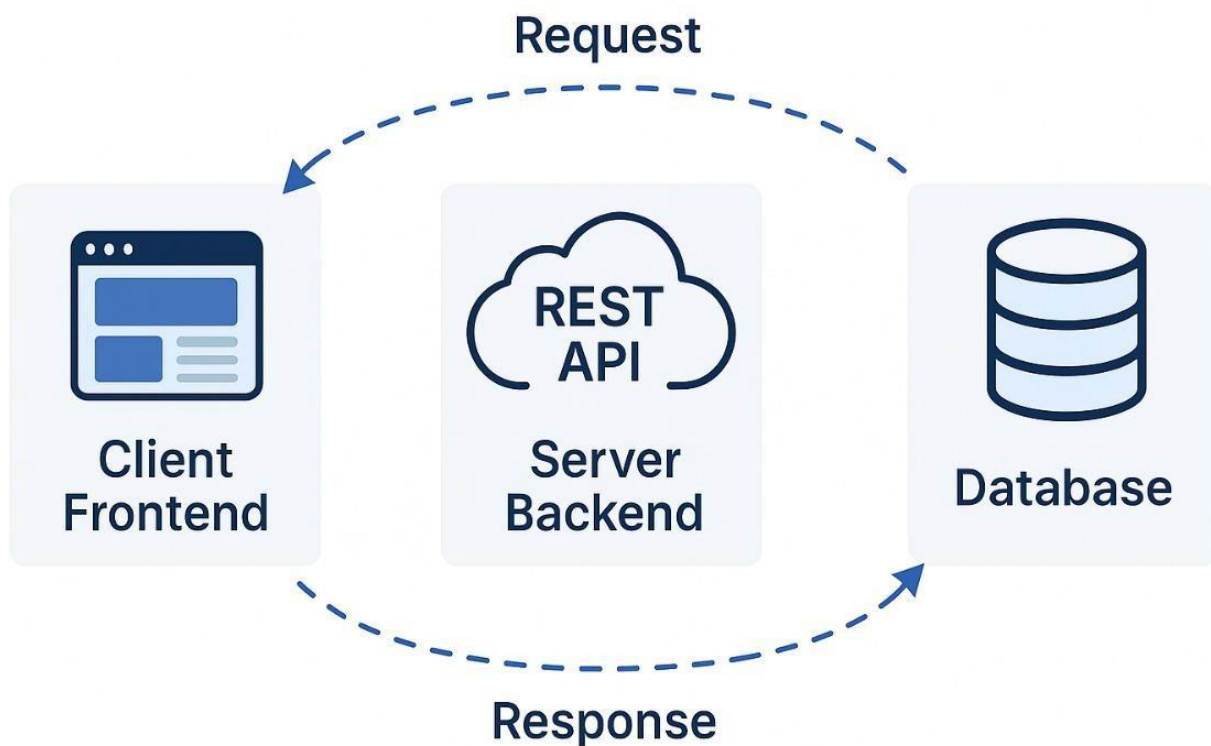


Fig 8.1 REST API Flow Between Frontend and Backend Tools

Introduced:

- Postman for API testing
- Axios / Fetch API for frontend requests
- Django REST Framework / Flask-RESTful
- JSON Formatter for debugging responses

Real-World Applications:

- Fetching weather, news, or product data
- User authentication and session management
- Submitting forms and storing data in databases
- Building dashboards and analytics tools

WEEK 9

Version Control with Git and Deployment Strategies

Topic Covered: Code Management, Collaboration, and Making Projects Live

Description:

This week focuses on two essential aspects of modern development:

- Version control using Git
- Deployment strategies for hosting full stack applications

Participants learn how to track changes, collaborate using Git and GitHub, and deploy projects to live environments using platforms like Netlify, Heroku, and Render. This ensures code is well-managed, recoverable, and accessible to users worldwide

Key Areas covered include:

Git Fundamentals

- What is Git and why use it
- Repositories, commits, branches, merges
- Common commands: git init, git clone, git add, git commit, git push, git pull

GitHub Collaboration

- Creating and managing remote repositories
- Pull requests and code reviews
- Branching strategies (feature, main, hotfix)

Deployment Platforms

- Hosting frontend with GitHub Pages, Netlify
- Hosting backend with Heroku, Render
- Environment variables and production settings

CI/CD Concepts

- Introduction to Continuous Integration and Deployment
- Automating builds and updates

Security and Maintenance

- SSH key and access control
- Monitoring logs scaling apps

Advantages of Git and Deployment:

- **Version History**-Track every change and revert when needed
- **Team Collaboration**-Multiple developers can work on the same project without conflict
- **Live Hosting**-Share your project with users, recruiters, or clients
- **Automation**-CI/CD pipeline streamline testing and deployment
- **Security and Control**-GitHub permissions, SSH keys, and environment configs protect your codebase

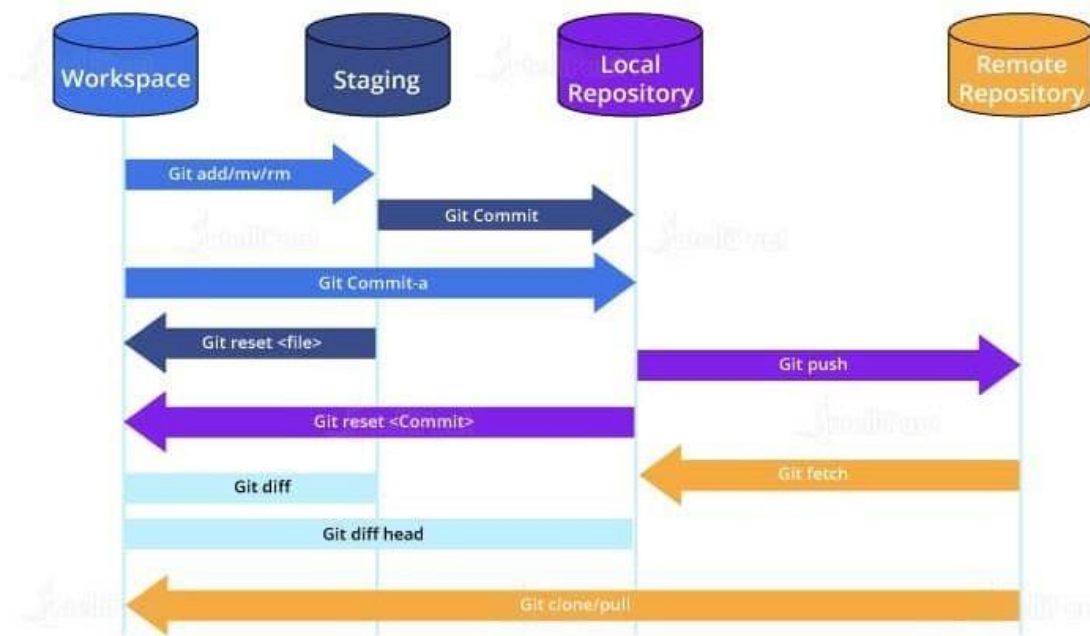


Fig. 9.1 Git Workflow and Deployment Pipeline Tools

Introduced:

- Git/GitHub for version control
- Netlify/GitHub Pages for frontend hosting
- Heroku/Render for backend deployment
- Postman/Logs for testing and monitoring
- CI/CD Pipelines (basic overview)

Real-World Applications:

- Hosting portfolios and client projects
- Managing codebases for teams
- Automating updates and bug fixes
- Sharing live demos and MVPs

WEEK 10

Capstone Project Implementation and Final Recap

Topic Covered: Applying Full Stack Skills in a Real-World Project

Description:

The final week of the internship is dedicated to building and presenting a capstone project that

integrates all the skills learned throughout the program. This hands-on experience allows participants to design, develop, and deploy a complete full stack application—from frontend UI to backend logic and database integration.

Participants also reflect on their learning journey, document their project, and prepare for real-world deployment and presentation.

Key Areas Covered:

Project Planning

- Defining project goals and scope
- Choosing tech stack and architecture

Frontend Development

- Designing responsive UI using HTML, CSS, JavaScript, and React

Backend Development

- Implementing server-side logic with Django or Flask
- Creating RESTful APIs

Database Integration

- Connecting to MySQL or MongoDB
- Performing CRUD operations

Deployment

- Hosting frontend and backend
- Using GitHub, Netlify, Heroku, or Render

Documentation and Presentation

- Writing README files and user guides
- Demonstrating project features and code flow

Advantages of Capstone Projects:

- **Portfolio-Ready Output**-Demonstrates practical skills to recruiters and mentors
- **End-to-End Experience**-Reinforces understanding of full stack workflows
- **Problem-Solving Practice**-Encourages debugging, iteration, and creative thinking
- **Confidence Building**-Validates your ability to build real-world applications
- **Team Collaboration (if applicable)**-Simulates industry-style teamwork and version control

Architecture of Full Stack Web Application

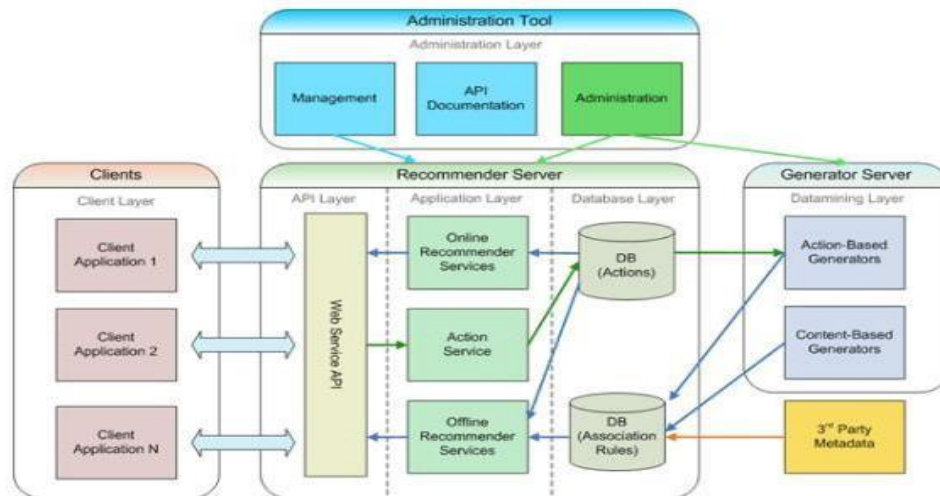


Fig 10.1 Capstone Project Architecture and Deployment Flow

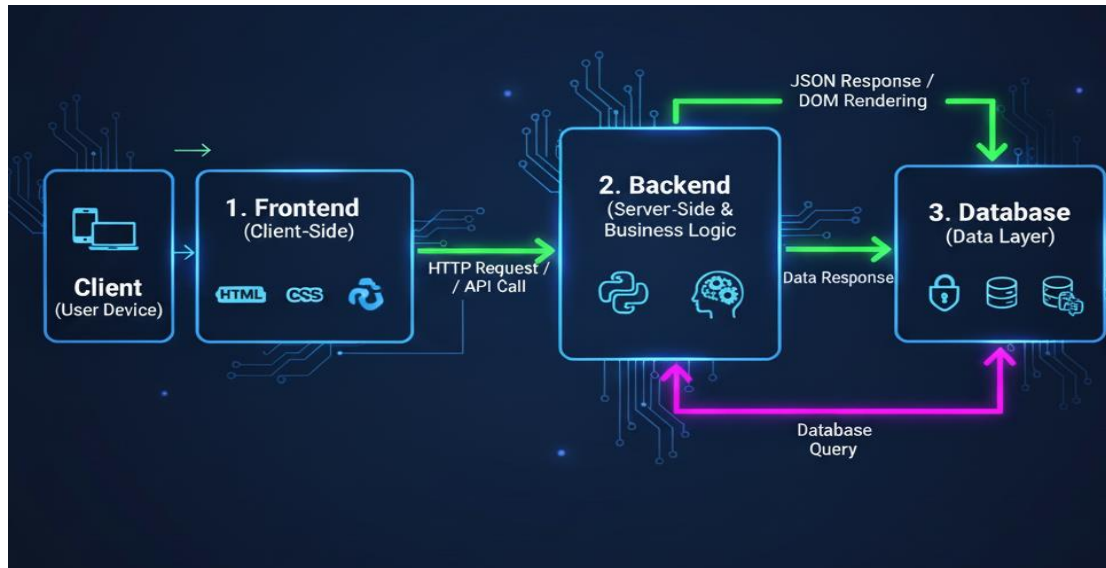
Tools Introduced:

- GitHub Projects / Issues for task tracking
- Markdown for documentation
- Deployment platforms (Heroku, Netlify, Render)
- Presentation tools (Power point, Loom, or live demo)

Real-World Applications:

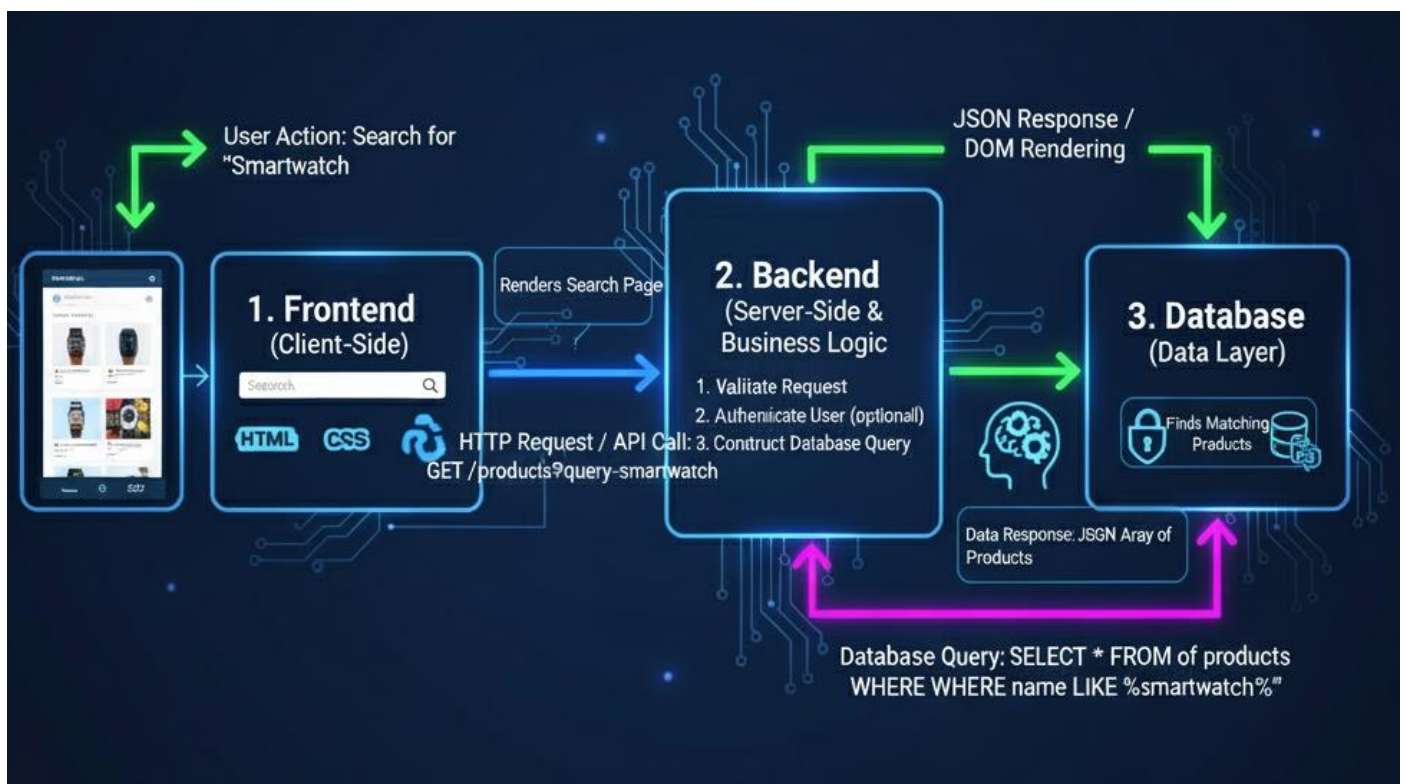
- Building a personal portfolio site
- Creating a task manager, blog, or e-commerce prototype
- Developing dashboards or academic portals
- Launching MVPs for startup id

Flow of Python Full Stack Developer



Example:

Application of Bookin the product on online



RESULT & DISCUSSIONS

TECHNOLOGY STACK

1. Frontend (Client-Side) Technologies

The technologies responsible for the User Interface (UI) and all direct user interaction in the browser.

Core Languages: HTML5, CSS3, JavaScript (ES6+).

Framework/Library

Styling: : React (or Next.js/Vue.js) for dynamic and component-based user interfaces (UIs). Tailwind CSS or Styled Components for rapid, utility-first styling and responsive design.

API Communication: Axios or the native Fetch API for making asynchronous calls (GET, POST, etc.) to the Backend.

2. Backend (Server-Side) Technologies

The engine that handles business logic, security, routing, and data processing.

Core Language: Python (excellent for data processing/microservices) or Node.js (JavaScript for performance and shared language).

Frameworks: Django (Python) or Express.js (Node.js) for building APIs and handling routing.

API Format: RESTful API design for standard, stateless communication between the Frontend and Backend.

Security: JWT (JSON Web Tokens) for user authentication and authorization.

3. Database (Data Layer) Technologies

The long-term storage and retrieval system for all application data (inventory, orders)

Conclusion

To summarize, my internship experience was a highly successful transition from academic theory to the professional reality of **Python Full-Stack Development**. The entire training was systematic, focused, and provided crucial industrial exposure, emphasizing that being a developer involves more than just writing code; it means understanding the full production lifecycle. I dedicated myself to mastering the projects, which taught me how to effectively use Python frameworks to build secure **Backend** logic and establish robust data connections.

This experience didn't just equip me with strong technical skills for both front-end and back-end integration; it instilled the professional discipline required to navigate a systematic workplace. I conclude that I am now a confident, well-rounded **Python Full-Stack Developer**, ready to immediately contribute end-to-end solutions to any professional team.

Student Self Evaluation of the Summer Internship

Student Name: HEMA LAKSHMI BOKKA

Registration No: 226N1A0512

Term of Internship:

From: Apr 2025 To : June 2025

Date of Evaluation:

Organization Name & Address:

Please rate your performance in the following areas:

Rating Scale:

Letter grade of CGPA calculation to be provided

1	Oral communication	1	2	3	4	5
2	Written communication	1	2	3	4	5
3	Proactiveness	1	2	3	4	5
4	Interaction ability with community	1	2	3	4	5
5	Positive Attitude	1	2	3	4	5
6	Self-confidence	1	2	3	4	5
7	Ability to learn	1	2	3	4	5
8	Work Plan and Organization	1	2	3	4	5
9	Professionalism	1	2	3	4	5
10	Creativity	1	2	3	4	5
11	Quality of work done	1	2	3	4	5
12	Understanding the Community	1	2	3	4	5
13	Achievement of Desired Outcomes	1	2	3	4	5
14	Time Management	1	2	3	4	5
15	OVERALL PERFORMANCE					

Date:

Signature of the Student:

MARKS STATEMENT
(To be used by the Examiners)

INTERNAL ASSESSMENT STATEMENT

Name of the Student : HEMA LAKSHMI BOKKA

Program of Study : B-TECH

Group : CSE

Roll Number : 226N1A0512

College Name : Srinivasa Institute of Engineering and Technology

Sl.No	Evaluation Criterion	Maximum Marks	Marks Awarded
1.	Activity Log	25	
2.	Internship Evaluation	50	
3.	Oral Presentation	25	
	GRAND TOTAL	100	

Date:

Signature of the Faculty of the Guide